Assignment = 1

1. Explain the difference between frontend, backend, and full- stack development with suitable real-world examples.

   Frontend development is the part of web development that users can see and interact with. It focuses on designing the layout, buttons, menus, and other visual elements of a website. Common technologies include HTML, CSS, and JavaScript.

   Example: The homepage, search bar, and shopping cart of an online shopping website.

   Backend Development:

   Backend development is the part that works behind the scenes. It manages the server, database, and application logic. It stores and retrieves data and ensures the website functions correctly. Common technologies include Java, Python, PHP, Node.js, and MySQL.

   Example: Checking a user's login details and processing online orders.

   Full-Stack Development:

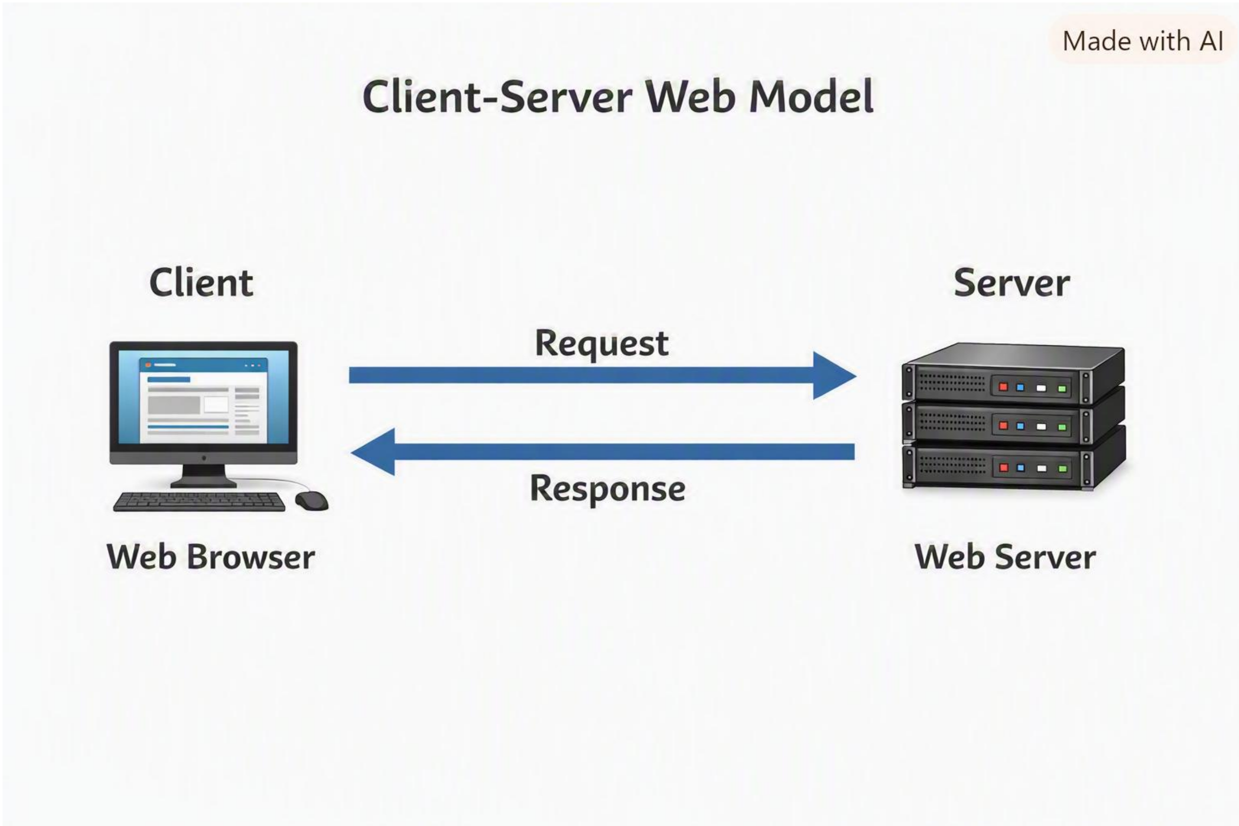
   Full-stack development combines both frontend and backend development. A full-stack developer can build an entire website or web application from the user interface to the server and database.

   Example: Creating a complete food delivery app with customer pages, restaurant management, and payment processing.

   Difference in Simple Words:

- **Frontend:** Works on what users see.
- **Backend:** Works on how the website functions behind the scenes.
- **Full-Stack:** Works on both frontend and backend to build a complete web application.

2. Create a simple diagram showing how the client-server model works in web architecture.



3. Describe how a browser requests and displays a web page from a web server.

When a user enters a website address (URL) in a browser and presses Enter, the browser sends an **HTTP request** to the web server. The web server receives the request, processes it, and retrieves the required files or data from the server or database. The server then sends an **HTTP response** containing the web page files (HTML, CSS, JavaScript, images, etc.) back to the browser. Finally, the browser reads these files, renders the content, and displays the complete web page on the user's screen.

The user enters a URL in the browser.

The browser sends an HTTP request to the web server.

The web server processes the request and retrieves the required data.

The server sends an HTTP response to the browser.

The browser renders the HTML, CSS, and JavaScript and displays the web page.

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Tool	Purpose
Visual Studio Code (VS Code)	Used to write and edit code.
Web Browser (Chrome/Firefox)	Used to test and view web pages.
Live Server	Automatically refreshes the web page after code changes.
Node.js	Runs JavaScript outside the browser and installs packages.
Git	Tracks code changes and manages versions.
GitHub	Stores and shares project code online.
npm (Node Package Manager)	Installs and manages libraries and packages.

5. Explain what a web server is and give examples of commonly used servers.

A **web server** is a software or computer that stores website files and delivers them to users' browsers when they request a web page using HTTP or HTTPS.

Commonly Used Web Servers:

- **Apache HTTP Server** – One of the most popular open-source web servers.
- **Nginx** – Known for its high performance and speed.

- **Microsoft IIS (Internet Information Services)** – A web server developed by Microsoft for Windows.
- **LiteSpeed** – A fast and secure web server commonly used for hosting websites.

6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

A frontend developer designs and develops the user interface of a website. They create the layout, buttons, menus, and other elements that users interact with using HTML, CSS, and JavaScript.

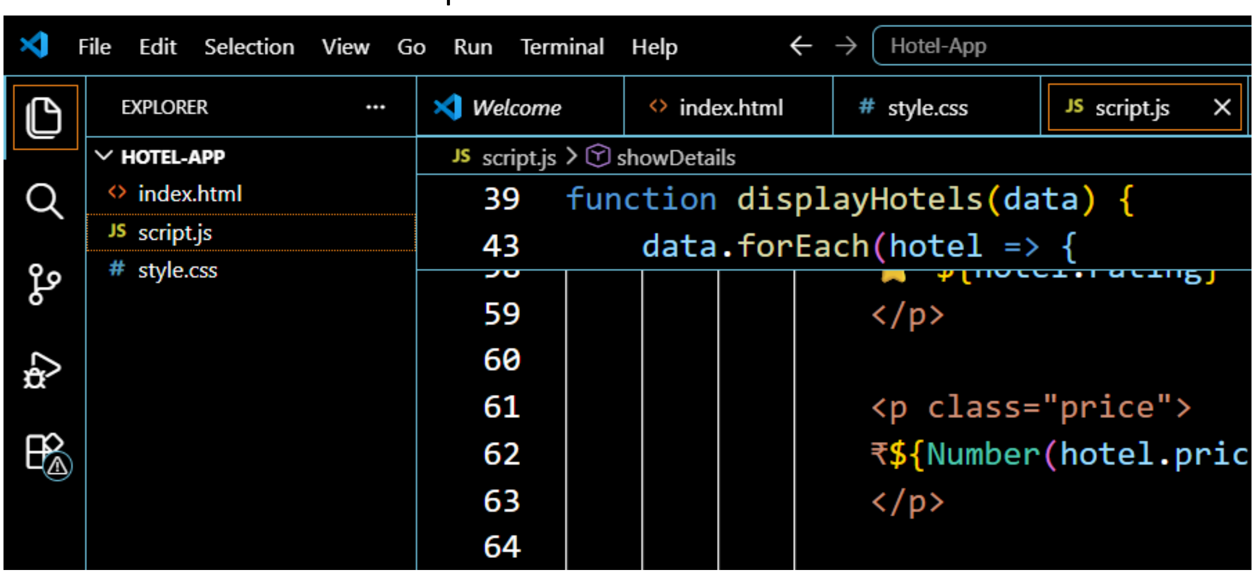
Backend Developer:

A backend developer builds the server-side logic of the website. They manage data processing, user authentication, and communication between the server and the database.

Database Administrator (DBA):

A database administrator manages and maintains the database. They ensure data is stored securely, organized properly, and can be accessed efficiently.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.



8. Difference Between Static and Dynamic Websites

Static Website:
A static website contains fixed content that does not change unless the developer manually updates the files. It is mainly built using HTML and CSS and provides the same information to every user.
Example: A personal portfolio website or a simple company information website.

Dynamic Website:
A dynamic website displays content that can change based on user interactions, database information, or other factors. It uses technologies like JavaScript, PHP, Python, or databases.
Example: Online shopping websites like Amazon or social media websites like Facebook.

9. Research and list five web browsers. Explain how rendering engines differ between them.

A **web browser** is an application used to access and display websites. It uses a **rendering engine** to interpret HTML, CSS, and JavaScript code and convert it into a webpage.

Five Common Web Browsers:

1. **Google Chrome** – Uses the **Blink** rendering engine. It provides fast and efficient webpage rendering.
2. **Mozilla Firefox** – Uses the **Gecko** rendering engine. It focuses on security, performance, and web standards.
3. **Microsoft Edge** – Uses the **Blink** rendering engine. It provides fast browsing and compatibility with modern websites.
4. **Safari** – Uses the **WebKit** rendering engine. It is optimized for Apple devices like Mac, iPhone, and iPad.

5. **Opera** – Uses the **Blink** rendering engine. It offers fast browsing and supports modern web features.

Difference Between Rendering Engines:
Rendering engines are responsible for displaying web pages. Different engines may interpret HTML, CSS, and JavaScript slightly differently, which can affect the appearance and performance of websites across browsers.

Conclusion:
Although browsers perform the same basic function, their rendering engines differ in speed, security, and compatibility with web technologies.

10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.